

NLP Final Task 2026

RAG-Based Mental Health Support Chatbot

Project Documentation

Team Members:

Mohamed Elmansy

Arwa AlaaEldin

Mariam Mohsen

1. Introduction

This document provides a comprehensive technical description of the RAG-Based Mental Health Support Chatbot, developed for the NLP Final Task 2026. The system is designed to offer grounded, empathetic, and context-aware conversational support to users seeking help with anxiety, depression, stress, and crisis-related concerns.

The architecture is composed of four interconnected NLP modules that run in sequence for every user message. The first module detects the language of the incoming message, the second identifies the user emotional state, the third determines the intent of the query, and the fourth retrieves relevant knowledge and generates a grounded response using a large language model.

The system is deployed locally through a FastAPI application that exposes a chat endpoint. The endpoint aggregates the outputs of all modules and returns a structured response containing the detected language, emotion, intent, and generated answer.

2. Project Structure

The project is organised into four module-specific notebooks and a deployment layer that integrates trained artefacts into a unified API.

```
NLP_Project/  
  modules/  
    01_language_detection.ipynb  
    02_clean_emotion_classifier.ipynb  
    03_intent_classifier.ipynb  
    04_rag_pipeline.ipynb  
  models/  
    language_detection_tfidf_linearsvc.pkl  
    language_detection_metrics.json  
    lid.176.ftz  
  data/  
    train.csv / valid.csv / test.csv  
    mental_health_counseling.csv  
  app.py  
  language_detection.py  
  emotion_classifier.py  
  intent_classifier.py  
  rag.py  
  models.py  
  config.py  
  requirements.txt  
  chat.html
```

The notebooks are self-contained development environments. Once a model is trained and saved, the deployment scripts load the relevant artefacts and expose them through the API. The intent classifier does not produce a saved local model because it relies on few-shot prompting through a remote LLM at inference time.

3. System Architecture and Pipeline Flow

Every incoming message travels through the integrated architecture before a reply is returned. The routing logic determines whether the chatbot should activate the RAG pipeline or return a direct lightweight response.

Step	Module	Technology	Output
1	Language Detection	TF-IDF char n-grams + LinearSVC	Language code such as en, ar, or fr
2	Emotion Classifier	Transformer-based emotion model with mental-health rules	Emotion or mental-health state
3	Intent Classifier	Few-shot prompting via Groq LLM	Intent label for routing
4	Q&A RAG	Qdrant + Sentence Transformers + Groq LLM	Empathy-aware counselling answer

If the intent is asking_mental_health_question, the RAG pipeline is invoked. For greetings, goodbyes, gratitude, and out-of-scope messages, the system returns a direct response and translates it to the user language when needed.

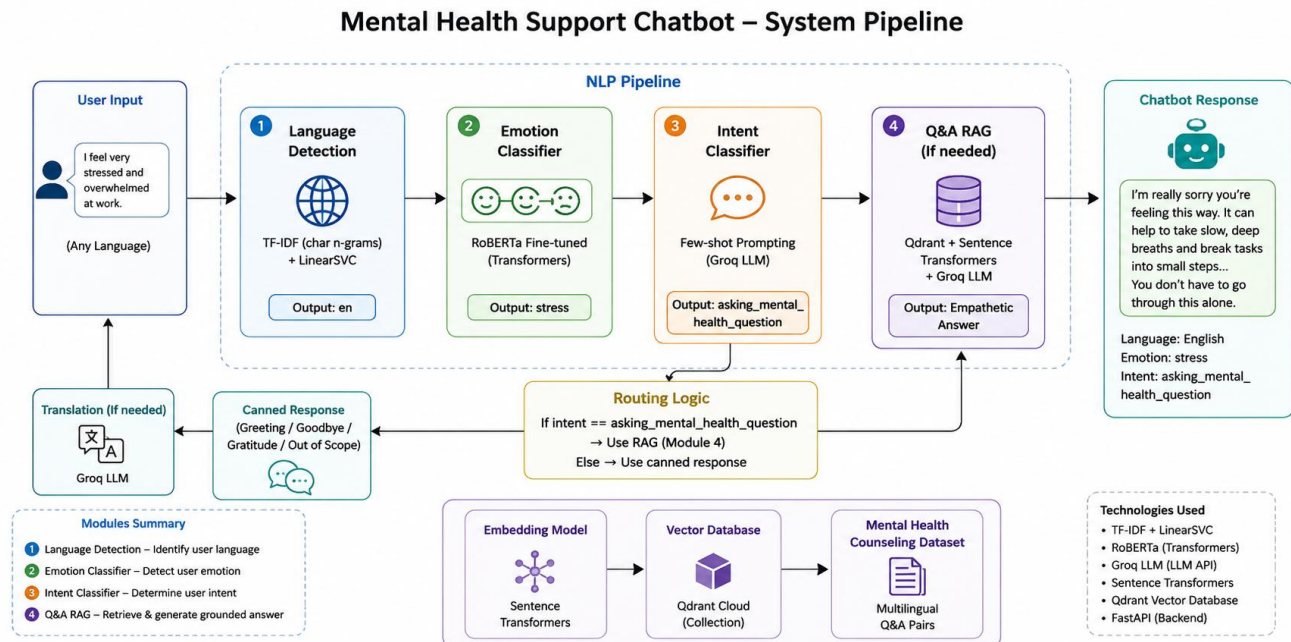


Figure 1. Full end-to-end architecture pipeline of the RAG-based mental health support chatbot.

4. Module 1 — Language Detection

4.1 Objective

The language detection module identifies the language of every user message before downstream processing. This information is required for multilingual response generation, translation control, and routing inside the chatbot.

4.2 Dataset

The module was trained on the papluca/language-identification dataset from Hugging Face. The dataset covers twenty languages and provides balanced train, validation, and test splits.

Split	Original Size	Augmented Size
Train	70,000	257,163
Validation	10,000	10,000 unchanged
Test	10,000	10,000 unchanged

The supported languages include Arabic, Bulgarian, German, Greek, English, Spanish, French, Hindi, Italian, Japanese, Dutch, Polish, Portuguese, Russian, Swahili, Thai, Turkish, Urdu, Vietnamese, and Chinese.

4.3 Challenges

4.3.1 Data Leakage

The first major challenge was data leakage. The dataset contained duplicate and near-duplicate samples across splits, which made early evaluation results look better than real application performance. This was especially visible when the model was deployed and tested with short real-user messages.

4.3.2 Poor Performance on Short Inputs

The original training samples were mostly long, fully formed sentences. In the real chatbot, users often write short messages, greetings, one-word inputs, or informal phrases. A character n-gram model has very limited evidence in these cases, so short-message predictions were unstable.

4.4 Approaches Tried and Rejected

4.4.1 Embedding-Based Deduplication

Sentence embeddings were used to calculate semantic similarity between training and test records. Near-duplicate examples were detected through cosine similarity and removed to reduce leakage. However, removing many similar records reduced the available training data and weakened the model on normal-length inputs.

4.4.2 TF-IDF Similarity Filtering

A second experiment used TF-IDF cosine similarity to remove highly similar samples between splits. This also reduced overlap but did not solve the core short-text problem, so it was not selected as the final approach.

4.5 Chosen Solution — Short-Text Data Augmentation

The final solution was controlled data augmentation applied only to the training split. For every long training sentence, short variants were generated, including the first two words, two middle words, the first three words, the first individual word, and the second individual word. Validation and test data remained untouched to prevent leakage.

This approach improved real application behavior because the model learned from both long and short language patterns. The final training set increased from 70,000 to 257,163 samples after augmentation and deduplication.

4.6 Model Architecture and Results

Pipeline	Validation Accuracy	Selected
TF-IDF char 2-5 grams + Logistic Regression	99.34%	No
TF-IDF char 2-5 grams + LinearSVC	99.43%	Yes

Metric	Value
Validation Accuracy	99.43%
Test Accuracy	99.55%
Number of Languages	20
Training Set Size after Augmentation	257,163 samples
Vectoriser	TF-IDF, character n-grams 2-5, 50k features
Classifier	LinearSVC

5. Module 2 — Emotion Classifier

5.1 Objective

The emotion classifier estimates the emotional state of the user message. This signal is passed to the response generation layer so the chatbot can adjust the tone of the answer.

5.2 Challenges

The main challenge was that generic emotion labels such as joy, sadness, anger, fear, love, and surprise are not always sufficient for mental-health conversations. Users frequently express anxiety, stress, panic, burnout, or emotional overload, which are more specific than the original six-label emotion dataset.

Short inputs such as wow, help, and ok also caused unstable predictions because the model was forced to select an emotion even when the message contained insufficient emotional context.

5.3 Solution Direction

The final application uses the transformer classifier as one component, but it is supported by translation for non-English inputs and rule-based mental-health overrides for short or high-risk phrases. This hybrid design is more suitable for a mental-health chatbot than relying on a generic emotion model alone.

Generic Label	Application-Level Meaning
joy	positive
sadness	sadness or depression-related tone
fear	anxiety or fear-related tone
anger	anger or frustration
love	positive or supportive tone
surprise	neutral or unclear depending on context

6. Module 3 — Intent Classifier

6.1 Objective

The intent classifier determines the purpose of the user message and controls the response route. It prevents unnecessary RAG calls for simple greetings, gratitude, goodbyes, or out-of-scope messages.

6.2 Approach

The classifier is implemented using few-shot prompting with a Groq-hosted large language model. The prompt defines the allowed labels and provides examples for each class. Temperature is set to zero to make the output deterministic.

- greeting — the user opens the conversation.
- goodbye — the user ends the conversation.
- gratitude — the user expresses thanks.
- asking_mental_health_question — the user asks for mental-health support.
- out_of_scope — the message is unrelated to the project scope.

7. Module 4 — Retrieval-Augmented Generation Pipeline

7.1 Objective

The RAG module answers mental-health questions using retrieved counselling examples instead of relying only on the LLM internal knowledge. This reduces hallucination and improves relevance.

7.2 Architecture

Component	Implementation
Vector Database	Qdrant Cloud
Embedding Model	all-MiniLM-L6-v2 Sentence Transformers
Large Language Model	openai/gpt-oss-120b via Groq API
Top-K Retrieval	3 most similar documents
Collection Name	mental_health_qa

The user query is translated to English for retrieval, embedded using the same sentence transformer used during indexing, and sent to Qdrant. The retrieved examples are injected into the final LLM prompt together with the detected emotion and language.

8. Deployment

8.1 Web Framework

The application is deployed using FastAPI. A POST endpoint at /chat accepts a JSON request containing the user message and returns language, emotion, intent, and answer.

8.2 Running the Application

1. Install dependencies using `pip install -r requirements.txt`.
2. Configure `GROQ_API_KEY`, `QDRANT_URL`, `QDRANT_API_KEY`, `EMOTION_MODEL_ID`, and `LANG_MODEL_PATH`.

3. Run the emotion model notebook and push the model to Hugging Face Hub if needed.
4. Run the RAG notebook to embed the counselling dataset and upload vectors to Qdrant.
5. Start the application with `uvicorn app:app --reload`.
6. Open the browser interface or use `/docs` to test the API.

8.3 API Response Format

```
{
  "message": "I feel very anxious and I cannot sleep at night",
  "language": "en",
  "emotion": "fear",
  "intent": "asking_mental_health_question",
  "answer": "It sounds like you are going through a really difficult time..."
}
```

9. Example Outputs and Image Handling

The following screenshots show the deployed browser interface and example chatbot outputs. The images were resized and placed separately to avoid overlap, cropping, or unreadable content in the final documentation.

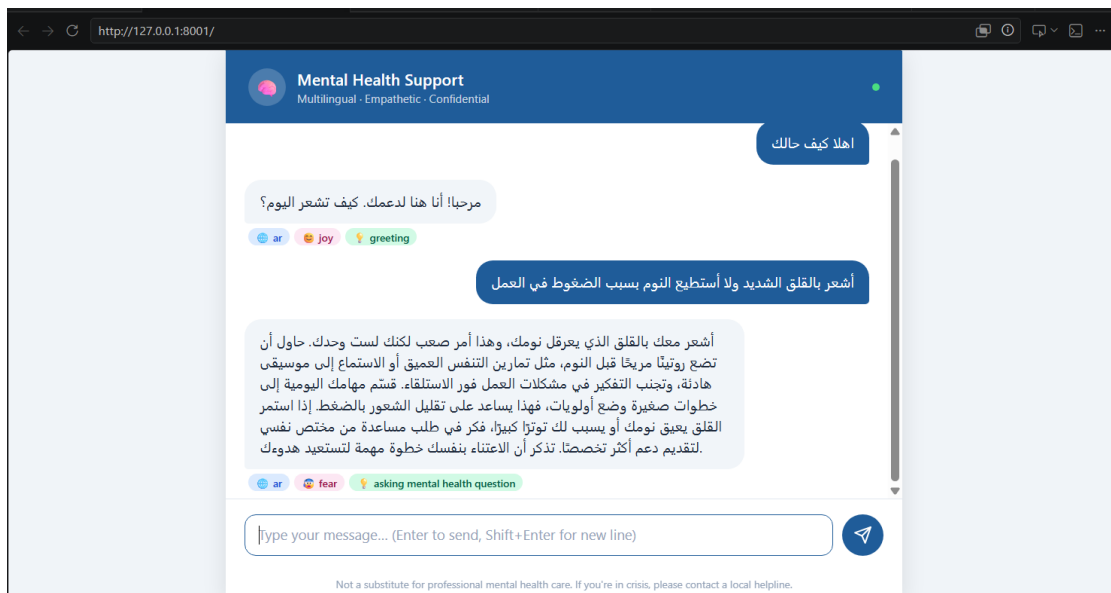


Figure 2. Example chatbot output screenshot 1.

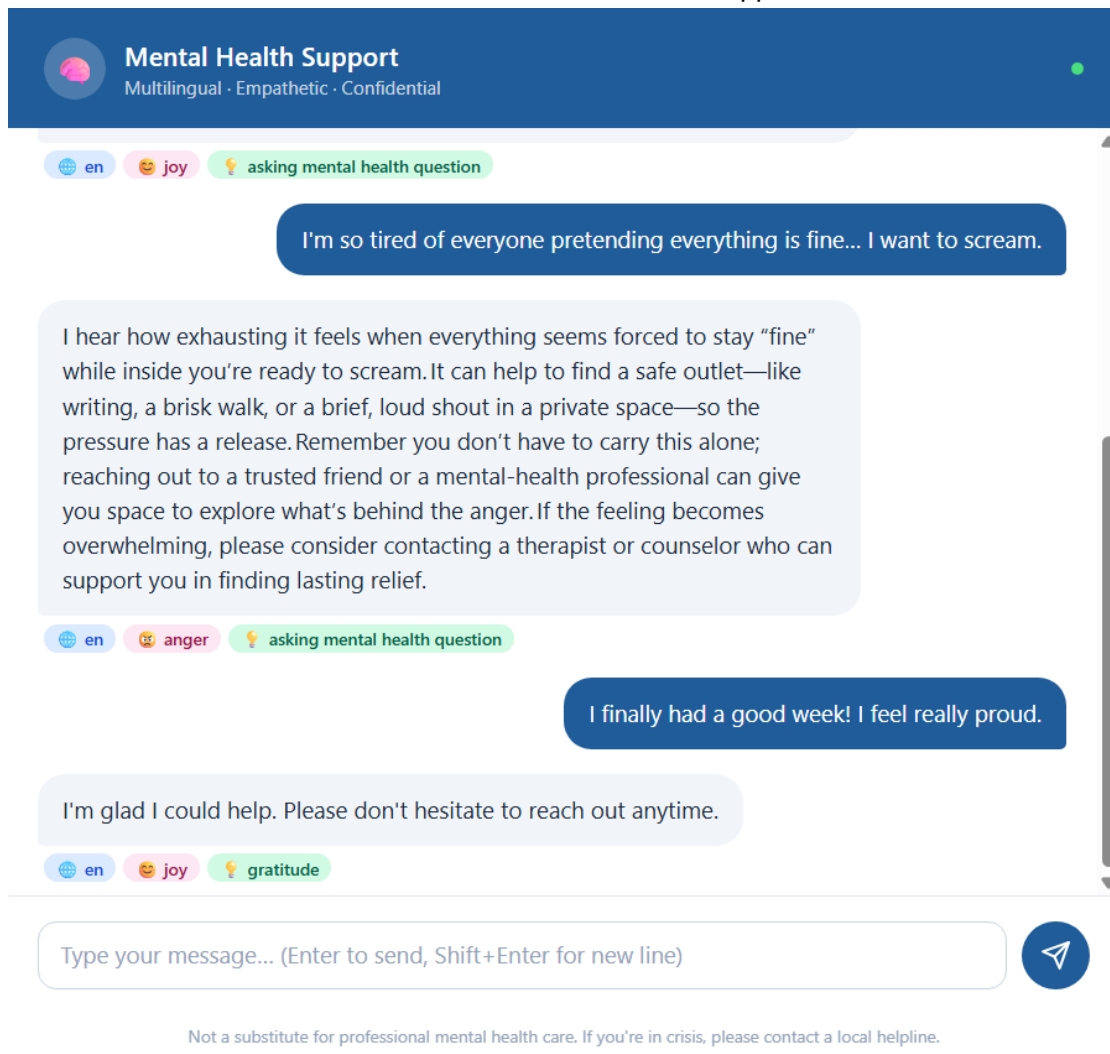


Figure 3. Example chatbot output screenshot 2.

NLP Final Task 2026 — Mental Health Support Chatbot

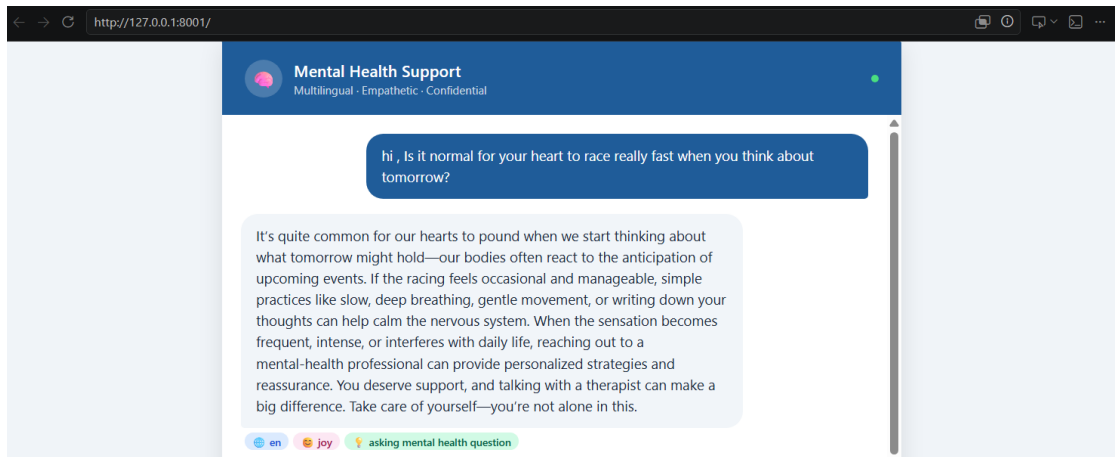


Figure 4. Example chatbot output screenshot 3.

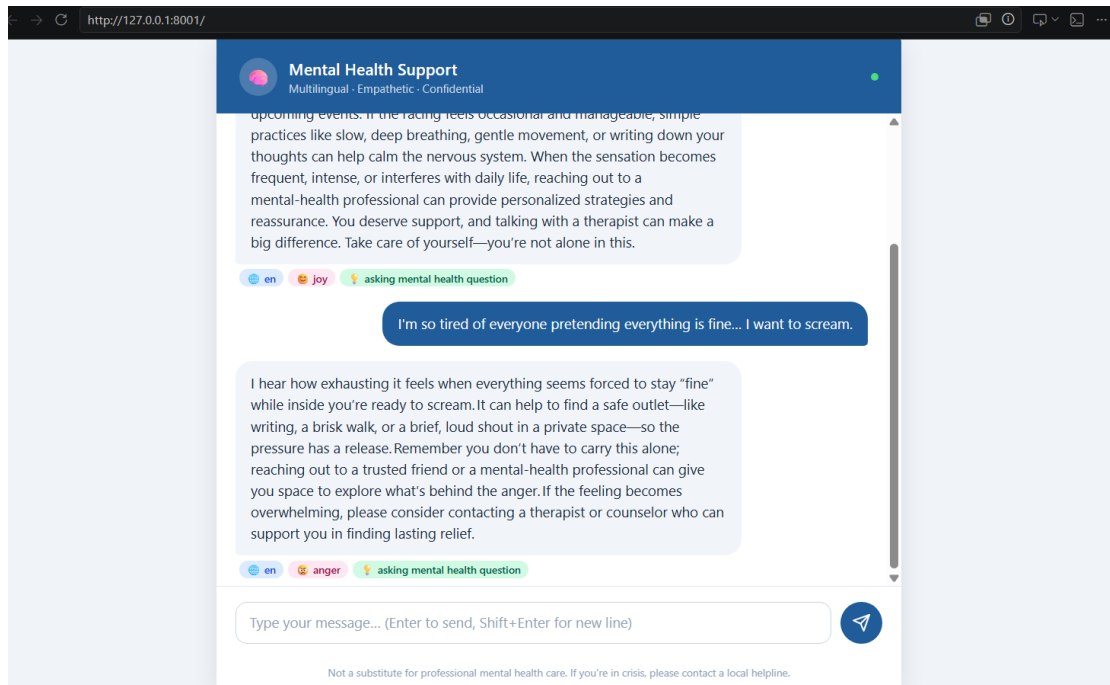


Figure 5. Example chatbot output screenshot 4.

10. Key Challenges and Solutions

Challenge	Impact	Solution
Data leakage in language detection	Inflated evaluation metrics and poor real-app performance	Train-only augmentation and untouched validation/test splits
Short-word language detection failure	Incorrect language predictions for one-word messages	Short-text augmentation and greeting dictionary fallback
Generic emotion labels	Stress and anxiety were mapped poorly to generic labels	Hybrid emotion system with mental-health rules
Multilingual retrieval	Non-English queries retrieved weaker English examples	Translate query to English for retrieval and respond in original language
Overusing RAG	Greeting and gratitude messages triggered unnecessary retrieval	Intent classifier routes only mental-health questions to RAG

11. Conclusion

The project demonstrates a complete multilingual mental-health support chatbot using a combination of traditional NLP, transformers, few-shot LLM prompting, vector search, and retrieval-augmented generation. The final architecture is modular, explainable, and suitable for local deployment through FastAPI.

The most important improvement came from evaluating the system under realistic user behavior rather than relying only on notebook-level metrics. This revealed short-text failures, emotion-label mismatch, and routing issues, which were addressed through hybrid machine-learning and rule-based solutions.